Event Processing Platform

System Design & SDLC Approach Document

Modernization of Legacy Notification System

Version 1.0 | March 2026 | CONFIDENTIAL

Property	Value
Document Owner	[Your Name]
Reviewed By	[Architecture Review Board]
Status	DRAFT — Pending Leadership Approval
Target Go-Live	[Q3/Q4 2026]
Tech Stack	Java 17, Spring Boot 3.x, Kafka, Oracle DB, Redis, OCP

Table of Contents

Table of Contents 2
1. Executive Summary 3
2. Problem Statement 3
3. Solution Overview 3
3.1 Architecture Summary 3
3.2 Key Design Principles 4
4. Phased Delivery Approach 5
Phase 0: Discovery & Legacy Analysis 5
Phase 1: Architecture & Detailed Design 6
Phase 2: Foundation & Spike 8
Phase 3: Core Implementation 8
Phase 4: Integration Testing & Performance 11
Phase 5: Deployment & Cutover 11
Phase 6: Hypercare & Legacy Decommission 13
5. Timeline Summary 13
6. Risk Assessment 14
7. Team Structure & Tooling 15
7.1 Team 15
7.2 Tooling 15
8. Success Criteria 15
9. Appendices 16

1. Executive Summary

This document presents the system design and phased delivery approach for modernizing our legacy notification/alerting platform into a scalable, event-driven architecture capable of processing 75,000+ messages per second across 15+ upstream sources.

The new platform replaces the existing monolithic system with a Kafka-based microservices architecture that is configuration-driven (new upstreams require zero code changes for simple integrations), resilient by default (circuit breakers, retry with exponential backoff, dead letter routing), and operationally observable (structured logging, distributed tracing, per-upstream SLA dashboards).

The approach leverages AI-assisted development (GitHub Copilot with Claude Sonnet) for both legacy business logic extraction and new system implementation, reducing the estimated development timeline from 16 weeks to 8–10 weeks while improving accuracy of business rule migration.

2. Problem Statement

The current legacy system faces the following challenges that necessitate modernization:

Scalability ceiling: The monolithic architecture cannot horizontally scale individual processing stages. Adding upstream #6 required 3 weeks of development and regression testing.

Tight coupling: Business logic for all upstreams is interleaved in shared classes. A change to Banking validation can break Insurance processing.

Limited resilience: Vendor API failures cause cascading delays across all upstreams. No circuit breakers, no automatic failover, no rate limiting.

Operational blind spots: No per-upstream SLA tracking, no structured error codes, no end-to-end message tracing. Debugging requires manual log searching across multiple systems.

Onboarding friction: Each new upstream requires custom development. The platform team is a bottleneck for business growth.

3. Solution Overview

The new Event Processing Platform is a Kafka-based, event-driven architecture decomposed into 3 microservices with a shared library, deployed on OpenShift. It ingests messages from multiple upstream sources, validates and normalizes them per upstream-specific rules, and dispatches notifications via SMS, Email, OTP, Voice, Webhooks, and Push channels through enterprise vendor APIs.

3.1 Architecture Summary

Component	Technology	Purpose

Ingestion Service	Spring Kafka, Redis	Kafka consumer, deduplication, schema validation, enrichment
Processor Service	Spring Kafka, Resilience4J, Oracle	Business validation, normalization, dispatch, retry, DLT
Audit Service	Spring Kafka, Oracle	Audit trail persistence, metrics, data archival
Platform Common	Java 17 library	Shared models, error codes, factory interfaces, Kafka config
Message Broker	Apache Kafka (Enterprise)	Event streaming, 10 partitions/topic, RF=3
Database	Oracle (RAC/Data Guard)	Event persistence, dispatch tracking, audit log
Cache	Redis Cluster	Dedup bloom filter, config cache, rate limit counters
Deployment	OpenShift (OCP)	Container orchestration, HPA, rolling updates

3.2 Key Design Principles

Principle	How It's Achieved
Zero message loss	Kafka acks=all, manual offset commit, transactional outbox pattern
Configuration-driven	New simple upstream = DB config row + Kafka topic. No code deployment.
Pluggable channels	Adapter pattern per channel. New channel = new @Component + config.
Resilient by default	Circuit breakers, rate limiters, bulkheads on every outbound call
Observable	Structured JSON logs, correlation ID tracing, per-upstream SLA metrics
Horizontally scalable	Stateless pods, Kafka partitioning, independent service scaling

4. Phased Delivery Approach

The project follows a hybrid Agile approach with clearly defined phases. Each phase has explicit inputs, outputs, acceptance criteria, and gates before proceeding to the next. AI-assisted tooling (GitHub Copilot) is integrated into discovery, design, and implementation phases to accelerate delivery without compromising quality.

Phase 0: Discovery & Legacy Analysis

Property	Detail
Duration	1–2 weeks
Team	1 Lead Architect + 1 Domain Expert per upstream
Objective	Extract and document ALL business logic from the legacy system before writing any new code

Inputs:

- Access to legacy codebase (read-only clone in VS Code workspace)
- GitHub Copilot Enterprise with Claude Sonnet model
- Domain experts available for 30-minute review sessions per upstream

Activities:

1. Index legacy codebase in VS Code for Copilot @workspace access.
2. Run AI-assisted extraction per upstream using structured prompts (one prompt per upstream with entry point file reference, automatic call graph tracing, config resolution, and rule extraction with confidence scoring).
3. Extract shared/common logic via call graph intersection across 3+ upstreams.
4. Run targeted searches for hidden logic (AOP aspects, servlet filters, DB triggers, Spring event listeners).
5. Domain expert reviews each spec (10–15 minutes per upstream).
6. Compile cross-reference matrix identifying shared rules, per-upstream complexity, and ConfigurableProcessor candidates.

Outputs (6 artifacts):

1. Per-upstream business logic specifications (one .md file per upstream)
2. Shared common logic specification (SHARED-COMMON-LOGIC.md)
3. Cross-reference matrix with upstream complexity classification
4. Migration debt register (dead code, tech debt, workarounds to drop or fix)
5. Upstream contract schemas (JSON Schema per upstream for CI validation)
6. Volume projections and current SLA baselines per upstream

Gate Criteria:

Every upstream spec is reviewed and signed off by the domain expert. Cross-reference matrix identifies which upstreams use ConfigurableProcessor (no code) vs custom processors (code required).

Phase 1: Architecture & Detailed Design

Property	Detail
Duration	1–2 weeks
Team	Lead Architect + Senior Developer
Objective	Finalize system architecture, DB schema, Kafka topology, error codes, and API contracts. Produce artifacts ready for AI-assisted implementation.

Inputs:

- Phase 0 outputs (all 6 discovery artifacts)
- Enterprise Kafka constraints (max partitions, replication factor, security)
- Oracle DB standards (naming, partitioning strategy, DBA review)
- OCP deployment standards (resource limits, namespace strategy, secrets management)
- Other team's draft design (if available) for gap reconciliation

Activities:

1. Finalize microservice boundaries and bounded contexts (3-service model).
2. Design Kafka topic layout (10-partition enterprise constraint, retention policies, partition key strategy).
3. Design Oracle schema (tables, sequences, indexes, partitioning strategy). DBA review.
4. Define complete error code taxonomy (25+ codes across all processing stages).
5. Design retry strategy (exponential backoff parameters, max retries per upstream, DLT routing).
6. Design Resilience4J configuration (circuit breaker, rate limiter, bulkhead per channel and upstream).
7. Produce architecture diagram (component + infrastructure view) and workflow diagram (message lifecycle).
8. Create .github/copilot-instructions.md with all project conventions for AI-assisted development.
9. Reconcile with other team's design: adopt what's better, document conflicts with rationale.

Outputs (5 key artifacts for development):

1. ARCHITECTURE.md — Complete system design with Kafka topology, DB schema, service boundaries
2. Architecture diagram (Mermaid) — Component + infrastructure view with labelled protocols
3. Workflow diagram (Mermaid) — Message lifecycle with all error codes at decision points
4. SQL DDL scripts — Oracle schema (V1 core tables, V2 outbox, V3 metering)
5. Error code enum (Java) — ErrorCode.java with stage, description, retryable flag

Gate Criteria:

Architecture review board approves design. DBA approves schema. Security team approves Kafka SASL/TLS and PII encryption approach.

Phase 2: Foundation & Spike

Property	Detail
Duration	1 week
Team	2 Developers
Objective	Set up project skeleton, shared library, CI/CD pipeline. Validate architecture with a thin end-to-end spike.

Inputs:

- Phase 1 outputs (5 architecture artifacts)
- Phase 0 outputs (legacy specs for the spike upstream)

Activities:

1. Create Git repository with Gradle multi-module structure (platform-common, 3 services).
2. Implement platform-common module: error codes, models, factory interfaces, Kafka config, async config.
3. Set up CI pipeline (build, test, SonarQube, container image build).
4. Run architecture spike: one message flowing from Kafka inbound through ingestion, processing, dispatch (mock vendor), and audit—using real Kafka (embedded), real Oracle (Testcontainers), and JDBC batch writes.
5. Validate: batch consumer works, outbox pattern works, JDBC batch INSERT performs, transactional producer commits correctly on Java 17 + Spring Boot 3.2 + Oracle.

Outputs:

- Compilable project skeleton with all modules
- Green CI pipeline
- Passing spike integration test (one message end-to-end)
- Spike findings document (any architecture adjustments needed)

Gate Criteria:

Spike test passes. No architecture changes required (or changes are documented and approved).

Phase 3: Core Implementation

Property	Detail
Duration	3–4 weeks

Team	3–4 Developers (parallelizable)
Objective	Implement all services, upstream processors, channel adapters, retry logic, and audit pipeline.

Inputs (fed to Copilot Plan + Agent mode):

- 5 architecture artifacts from Phase 1
- Per-upstream business logic specs from Phase 0
- .github/copilot-instructions.md (project conventions)
- Passing spike from Phase 2 (proven patterns)

Implementation sub-phases (sequential within, parallelizable across):

Sub-Phase	Scope	Duration	Parallel?
3A. Ingestion Service	Kafka listener, dedup (Redis+DB), schema validation, enrichment	3–4 days	No (foundation)
3B. Processing Layer	Upstream processors (from legacy specs), batch DB persist, outbox	5–7 days	After 3A
3C. Dispatch Layer	Channel adapters (SMS, Email, OTP, Voice, Webhook, Push), async thread pool, Resilience4J, batch DB writes	5–7 days	Parallel with 3B
3D. Retry & DLT	Exponential backoff, max retry enforcement, DLT routing, vendor failover	2–3 days	After 3C
3E. Audit Service	Batch consumer, batch persist, metrics exporter, archival scheduler	2–3 days	Parallel with 3B
3F. ConfigurableProcessor	No-code upstream onboarding for simple upstreams (config-driven)	2–3 days	After 3B

AI-Assisted Development Workflow:

1. Developer opens Copilot Plan mode, feeds architecture artifacts + relevant upstream spec.
2. Copilot generates implementation plan for the sub-phase.
3. Developer switches to Agent mode, implements one sub-phase at a time using #file references for precision.
4. TDD approach: Copilot generates tests from spec first, then implementation to pass tests.
5. Each sub-phase must compile and pass tests before the next begins.

Gate Criteria:

All sub-phases compile independently. Unit tests pass. Integration tests pass per service (embedded Kafka + Testcontainers).

Phase 4: Integration Testing & Performance

Property	Detail
Duration	1–2 weeks
Team	2 Developers + 1 QA
Objective	End-to-end integration tests, performance/load testing, chaos testing.

Test Matrix:

Test Type	Scope	Tools	Pass Criteria
Integration	Full pipeline: Kafka → Ingestion → Processing → Dispatch → Audit	Testcontainers (Oracle, Redis), Embedded Kafka, WireMock	1000 messages processed, 0 lost, audit trail complete
Dedup	Send 1000 messages, re-send same 1000	Same as above	DB count = 1000, not 2000
Retry/DLT	Mock vendor returning 500 for N messages	WireMock + assertions	Failed messages retry 3x, then land in DLT
Contract	Validate upstream schemas against processors	JSON Schema Validator	All contracts pass, schema changes caught
Performance	75K msg/s sustained for 10 minutes	Custom load generator + Prometheus/Grafana	P99 latency < 2s, consumer lag < 10K, no OOM
Chaos	Kill consumer pod mid-batch, kill DB connection mid-write	Pod kill scripts, network policy rules	No message loss, consumers recover within 45s

Gate Criteria:

All test types pass. Performance test meets NFR targets. Chaos test confirms zero message loss. Load test report reviewed by architect.

Phase 5: Deployment & Cutover

Property	Detail
Duration	1–2 weeks
Team	1 Developer + 1 DevOps + 1 DBA
Objective	Production deployment, parallel-run validation, and phased upstream cutover.

Deployment Strategy:

1. Deploy to staging environment. Run full integration + performance test suite.

2. Deploy to production with consumers PAUSED (zero traffic).
3. DBA executes DDL scripts (V1, V2, V3). Validates partitioning, indexes, sequences.
4. DevOps creates Kafka topics (create-kafka-topics-enterprise.sh).
5. Parallel run: route upstream #1 (lowest risk) to BOTH legacy and new platform. Compare outputs for 48 hours.
6. If parallel run passes: cut over upstream #1 to new platform only. Monitor for 1 week.
7. Progressive cutover: one upstream per week, ordered by risk (lowest first).

Cutover Sequence (recommended):

Week	Upstream	Risk Level	Rationale
Week 1	Logistics	Low	Lowest volume, webhook-only, simplest validation
Week 2	Marketing	Low	Bulk messaging, tolerant of latency, no regulatory constraints
Week 3	Insurance	Medium	Moderate rules, single channel, IRDAI compliance
Week 4	Banking	High	High volume, multi-channel, RBI compliance, OTP critical path
Week 5	OTP Auth	High	Time-critical, highest priority, zero-tolerance for failure

Rollback Plan:

Each upstream cutover is independently reversible. If the new platform shows degraded delivery rates or latency exceeding SLA for any upstream, that upstream is routed back to the legacy system within 15 minutes via Kafka topic reconfiguration. No code deployment required for rollback.

Gate Criteria:

48-hour parallel run shows zero message loss and delivery rate within 0.5% of legacy. Per-upstream cutover is approved by the upstream business owner.

Phase 6: Hypercare & Legacy Decommission

Property	Detail
Duration	2–4 weeks (overlaps with cutover tail)
Team	1 Developer (on-call) + DevOps
Objective	Post-cutover monitoring, SLA validation, legacy system decommission.

Activities:

1. Monitor per-upstream SLA dashboards daily (delivery rate, P99 latency, DLT rate, error distribution).
2. Tune Resilience4J parameters based on production vendor behavior (circuit breaker thresholds, rate limits).
3. Validate data archival jobs are running correctly (monthly partition management).
4. After all upstreams stable for 2 weeks: decommission legacy system consumers (keep legacy DB read-only for 90 days).
5. Retrospective and lessons-learned document.

Gate Criteria:

All upstreams meeting SLA targets for 2+ consecutive weeks. Legacy system decommissioned. Retrospective completed.

5. Timeline Summary

Phase	Duration	Key Milestone
Phase 0: Discovery & Legacy Analysis	1–2 weeks	All upstream specs extracted and reviewed
Phase 1: Architecture & Design	1–2 weeks	Architecture approved by review board
Phase 2: Foundation & Spike	1 week	E2E spike passing, CI/CD green
Phase 3: Core Implementation	3–4 weeks	All services implemented and unit tested
Phase 4: Integration & Performance	1–2 weeks	75K msg/s sustained, chaos test passed
Phase 5: Deployment & Cutover	1–2 weeks per upstream	All upstreams on new platform
Phase 6: Hypercare & Decommission	2–4 weeks	Legacy decommissioned, SLA met
TOTAL	10–16 weeks	Full production cutover

6. Risk Assessment

Risk	Impact	Likelihood	Mitigation
Legacy rules missed during extraction	High	Medium	Domain expert review + contract tests in CI + 48-hour parallel run validates equivalence
Oracle performance at 75K msg/s	High	Low	JDBC batch writes, sequence sharding, performance test in Phase 4 with production-sized DB
Kafka 10-partition limit insufficient	Medium	Low	Topic-per-upstream for processing at scale. Current design handles 75K msg/s on 10 partitions.
Vendor API latency spikes during cutover	Medium	Medium	Circuit breakers + vendor failover chain + per-vendor bulkheads
Team unfamiliar with Kafka patterns	Medium	Medium	Hackathon challenge pre-cutover + spike in Phase 2 builds team confidence
Copilot generates incorrect business logic	Medium	Low	TDD approach (tests first from spec, then implementation). Contract tests catch drift.

7. Team Structure & Tooling

7.1 Team

Role	Count	Phases Active
Lead Architect	1	All phases
Senior Developer	1–2	Phase 1–6
Developer	2–3	Phase 2–5
QA Engineer	1	Phase 4–5
DevOps Engineer	1	Phase 2, 5–6
DBA	1 (part-time)	Phase 1, 2, 5
Domain Expert	1 per upstream	Phase 0 (30 min/upstream), Phase 5 (cutover approval)

7.2 Tooling

Tool	Purpose	Phase Used
GitHub Copilot (Claude Sonnet)	Legacy extraction, plan generation, AI-assisted implementation	Phase 0–3
VS Code + Copilot Agent mode	Code generation with #file precision and TDD flow	Phase 2–3
Testcontainers	Oracle XE + Redis in CI integration tests	Phase 2–4
WireMock	Mock vendor APIs for dispatch testing	Phase 2–4
Embedded Kafka	Kafka integration tests without external cluster	Phase 2–4
SonarQube	Code quality, coverage, security scanning	Phase 2–4
Prometheus + Grafana	Metrics, dashboards, SLA monitoring	Phase 4–6
ELK / Splunk	Log aggregation and structured log search	Phase 4–6

8. Success Criteria

Metric	Target	Measurement
Message throughput	≥75,000 msg/s sustained	Performance test (Phase 4)

End-to-end latency (P99)	<2 seconds	Prometheus metrics in production
Delivery rate (per upstream)	≥99.5%	upstream_metrics_hourly dashboard
Message loss	Zero	Chaos testing + parallel run comparison
DLT rate	<0.5% of total volume	dlt.events topic monitoring
Consumer restart recovery	<45 seconds	Static group membership + chaos test
New simple upstream onboarding	<1 day (config only)	ConfigurableProcessor demo
New complex upstream onboarding	<1 week (code + config)	Custom processor + CI
Legacy business rules accuracy	≥99%	Contract tests + parallel run delta

9. Appendices

The following supporting documents are maintained in the project repository and are referenced throughout this document:

Document	Location	Purpose
Architecture Design	docs/ARCHITECTURE.md	Detailed system design with Kafka topology and DB schema
Architecture Addendum	docs/ARCHITECTURE-ADDENDUM.md	Answers to partition limits, concurrency, async threads, batch DB
Gap Analysis	docs/GAP-ANALYSIS.md	15 missing patterns identified and solutions
Scale-Proofing Analysis	docs/SCALE-PROOFING.md	What breaks at 15 upstreams and how to prevent it
Workflow Diagram	docs/workflow-diagram.mermaid	Message lifecycle with all error codes
Architecture Diagram	docs/architecture-diagram.mermaid	Component + infrastructure view
Copilot Dev Guide	docs/COPILOT-DEV-GUIDE.md	AI-assisted development workflow and prompts
Extraction Prompts	docs/STREAMLINED-EXTRACTION.md	One-prompt-per-upstream legacy extraction approach
Legacy Spec Template	docs/legacy-specs/TEMPLATE-business-logic.md	Standardized format for upstream specs

SQL Schema	schema/V1, V2, V3	Oracle DDL scripts (core, outbox, metering)
Kafka Topics	config/create-kafka-topics-enterprise.sh	Topic creation with 10-partition constraint
OCP Deployment	config/ocp/	OpenShift deployment manifests and ConfigMaps
Copilot Instructions	.github/copilot-instructions.md	Auto-loaded project conventions for AI-assisted development